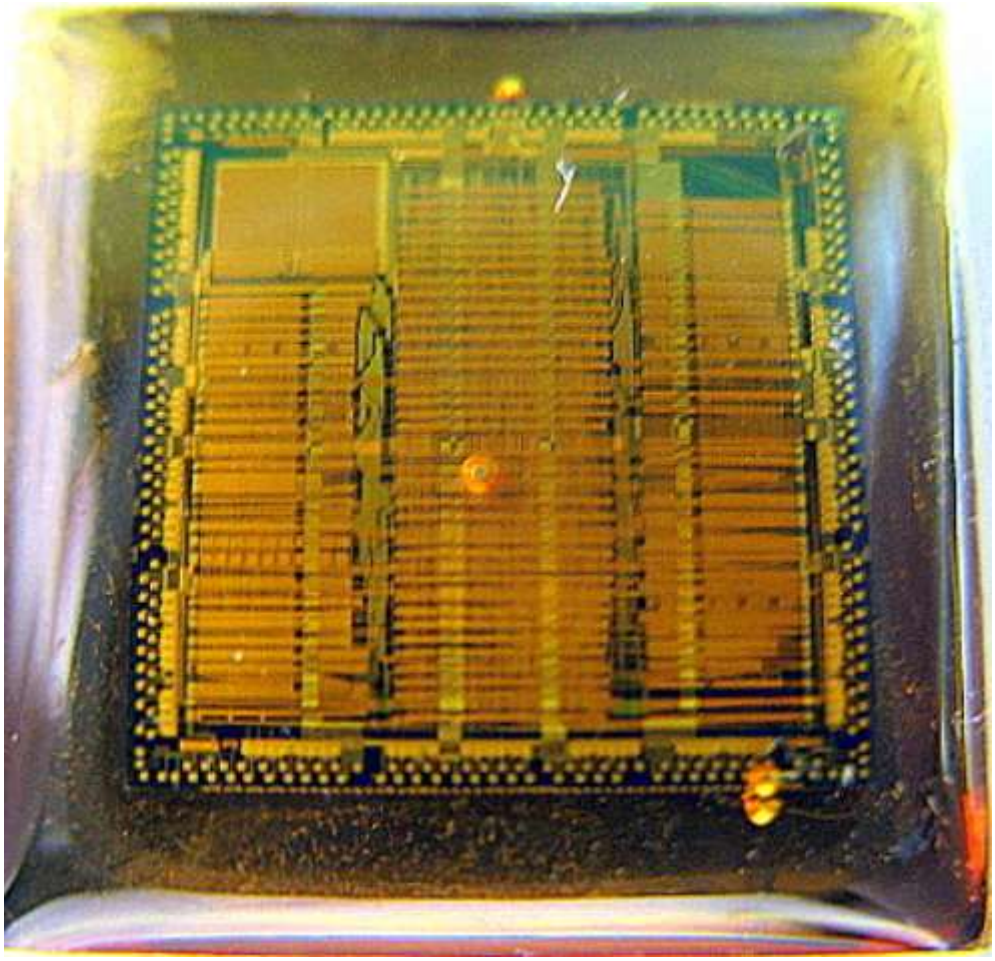
# Progettazione ed Implementazione di Un Processore MIPS 32bit Monociclo su *Open Silicon*.



Manuel D'Innocenti 282371

# Sommario

Introduzione: 4
*Istruction Set Architecture*: 5
Istruzioni di Tipo R: 6
Istruzioni di Shift: 6
Altre Eccezioni: 7
Istruzioni di Tipo I: 7
Istruzioni di Load e Store: 8
Istruzioni di Salto Condizionato: 8
Istruzioni di Tipo J: 9
Istruzioni Implementate: 10
Limitazioni di *TinyTapeout*: 12
Schema a Blocchi: 13
Dispositivo HOST: 13
Descrizione Generale: 14
Segnali della CONTROL_UNIT: 16
Funzionamento Istruzioni Standard di tipo “R”: 17
Istruzioni di *Shift*: 17
Funzionamento Istruzioni Standard di tipo “I”: 18
Funzionamento delle Istruzioni di Load e Store: 19
Funzionamento delle Istruzioni di Salto Condizionato: 20
Funzionamento delle Istruzioni di Tipo “J”: 21
Descrizione Dettagliata dei Blocchi: 22
*IO_CONTROLLER:* 22
Gestione delle Istruzioni Aritmetiche: 23
Gestione delle Istruzioni di salto e di *Store Word*: 23
Gestione della *Load Word:* 23
REGISTER_PORT: 24
SIMULAZIONE E TEST_BENCH: 25
Conclusioni: 29

Tabella 1: Campi dell'Istruzione in Relazione al suo Tipo ..... 5
Tabella 2: Illustrazione della disposizione dei bit nelle istruzioni di tipo "R"..... 6
Tabella 3: Esempio della disposizione di una ADD ..... 6
Tabella 4: Esempio della disposizione di una SLL..... 7
Tabella 5: Illustrazione della disposizione dei bit nelle istruzioni di tipo "I" ..... 7
Tabella 6: Esempio della disposizione di una ADDI ..... 7
Tabella 7: Illustrazione della disposizione dei bit nelle istruzioni di tipo "J"..... 9
Tabella 8: Esempio della disposizione di una J ..... 9
Tabella 9: Elenco delle istruzioni implementate in questo progetto .....11
Tabella 10: Pinout di TinyTapeout..... 12
Tabella 11: Descrizione dei segnali d'uscita della CONTROL_UNIT ..... 16
Tabella 12: Stato dei segnali d'uscita della CONTROL UNIT per istruzioni di tipo “R” ..... 17
Tabella 13: Stato dei segnali d'uscita della CONTROL UNIT per istruzioni di tipo "I" ..... 18
Tabella 14: Stato dei segnali d'uscita della CONTROL UNIT per istruzioni di LOAD/STORE ..... 19
Tabella 15: Stato dei segnali d'uscita della CONTROL UNIT per istruzioni di LOAD IMMIDIATE ..... 19
Tabella 16 Stato dei segnali d'uscita della CONTROL UNIT per istruzioni di salto condizionato..... 20
Tabella 17: Stato dei segnali d'uscita della CONTROL UNIT per istruzioni di salto incondizionato..... 21

# Introduzione:

Il MIPS (*Microprocessor without Interlocked Pipelined Stages*) è una famiglia RISC (Reduced Instruction Set Computer) di ISA (*Istruction Set Architecture*) sviluppato dalla *MIPS Computer Systems* (adesso MIPS Technologies) per la prima volta nel 1985.

Ad oggi lo sviluppo del MIPS è stato interrotto (ufficialmente in aprile 2021) in favore dello sviluppo di architetture RISC-V.

Le architetture MIPS hanno rappresentato la storia dei processori RISC e implementazioni di questa architettura possono essere trovate in oggetti che hanno rappresentato l'infanzia di moltissime persone (ad esempio Playstation 1 e 2, Nintendo 64 ecc…). È su questa vena di nostalgia che si è voluto resuscitare un'ultima volta questa architettura, oltre a voler dare un tributo al processore che (è qui mi svesto momentaneamente dal tono impersonale di questa tesina) mi ha fatto innamorare dell'elettronica digitale.

Un utile premessa è che in questo progetto **NON** vengono implementate le istruzioni del MIPS esteso. Bisogna tenere a mente che l'obiettivo è quella di questo progetto è quello di creare un'architettura *Touring completa* e per farlo non è necessario implementare tutte le istruzioni MIPS.

Questa tesina è organizzata nel seguente modo:

- Si introduce l'ISA di questa architettura e si discute quale sottoinsieme di istruzioni vengono implementate;

- Si discute delle limitazioni del circuito fisico prodotto su *TinyTapeout* e su come queste vadano a condizionare il progetto;

- L'analisi del sistema complessivo e dei sottosistemi che lo compongono;

- Una panoramica sulle simulazioni più importanti.

# *Istruction Set Architecture*:

Essendo il MIPS un'architettura RISC, quindi, l'istruzione macchina che viene elaborata dal processore ha una lunghezza fissa che in questo progetto è stata scelta di 32bit.

Esistono solo tre tipi di istruzioni nel MIPS base e sono:

- **Istruzioni di tipo R** (Istruzioni che modificano il contenuto di un registro prendendo i dati da un altro registro. Un esempio: **ADD $t3, $t1, $**t2, cioè, inserire nel registro $t3 la somma dei contenuti di $t1 e $t2);

- **Istruzioni di tipo I** (Istruzioni con *Immediato*, ovvero un numero fisso scritto direttamente nell'istruzione. Un esempio: **ADDI $t3, $t2, 20**, cioè, inserire nel registro $t3 la somma del contenuto in $t2 e 20);

- **Istruzioni di tipo J** (Istruzioni di salto non condizionato, Sono istruzioni che hanno come parametro solo un numero "immediato" come target. Come calcolare l'indirizzo target dipende dal tipo di istruzione e verrà discusso in seguito. Un esempio: **JUMP 250**, cioè, salta all'istruzione che ha come indirizzo di memoria 250).

A seconda del tipo d'istruzione i 32bit rappresentano sono quindi formattati in modo differente. Di seguito una tabella riassuntiva della mappa delle istruzioni.

| **Tipo di istruzione** | **Bit 31-26** | **Bit 25-21** | **Bit 20-16** | **Bit 15-11** | **Bit 10-6** | **Bit 5-0** |
|---|---|---|---|---|---|---|
| **Tipo-R** | opcode | rs | rt | rd | shamt | funct |
| **Tipo-I** | opcode | rs | rt | imm | imm | imm |
| **Tipo-J** | opcode | address | address | address | address | address |

*Tabella 1: Campi dell'Istruzione in Relazione al suo Tipo*

Per ragioni di chiarezza, di seguito vengono riportati i significati degli acronimi utilizzati:

- **Opcode (Operational Code):** Identifica il tipo di istruzione, se di tipo R questa è sempre ***"0"*** dando indicazione all'unità di controllo di andare a valutare il campo funzione. Nel caso di altri tipi d'istruzione, questo campo va a codificare il tipo di operazione da effettuare;

- **Funct:** Identifica univocamente il tipo di operazione "R" da eseguire;

- **RS (Source Register):** Identifica il registro che "subisce" l'operazione, ovvero, pensando ad un'operazione aritmetica, *RS* rappresenta l'operando che sta più a sinistra;

- **RT (Target Register):** Identifica il registro da cui si prende il secondo operando nelle istruzioni di tipo “R”, mentre in quelle di tipo “I” rappresenta il registro in cui viene salvato il risultato;

- **RD (Destination Register):** Identifica il registro in cui viene salvato il risultato di un’operazione;

- **Shamt (Shift Amount):** Identifica nelle operazioni di shift la quantità di bit che devono essere shift.

## Istruzioni di Tipo R:

| **Tipo di istruzione** | **Bit 31-26** | **Bit 25-21** | **Bit 20-16** | **Bit 15-11** | **Bit 10-6** | **Bit 5-0** |
|---|---|---|---|---|---|---|
| **Tipo-R** | opcode | rs | rt | rd | shamt | funct |

*Tabella 2: Illustrazione della disposizione dei bit nelle istruzioni di tipo "R"*

Le istruzioni di tipo R sono, in generale, istruzioni che operano sui registri seguendo uno schema del tipo:

$$RD = RS - operazione - RT$$

Ed il campo opcode e shamt vengono, nella maggior parte dei casi tenuti a ***“0”***.

*ESEMPIO:*

$$ADD\ \$t3, \$t1, \$t2$$

| opcode | rs | rt | rd | shamt | funct |
|---|---|---|---|---|---|
| 000000 | $t1 | $t2 | $t3 | 00000 | 100000 |

*Tabella 3: Esempio della disposizione di una ADD*

Esistono però delle eccezioni a questo flusso di esecuzione.

## Istruzioni di Shift:

Nelle istruzioni di Shift il campo shamt identifica il numero di bit che devono essere traslati, mentre il campo RS viene non considerato (normalmente si mette a ***“0”***).

Questo però **NON** vale per le istruzioni di tipo "*shift variable*" in cui il numero di bit da shiftare viene prelevato da un registro. In questi casi si segue il flusso classico delle istruzioni di tipo "R".ESEMPIO:

$$SLL\ \$t3, \$t2, 4$$

| opcode | rs | rt | rd | shamt | funct |
|---|---|---|---|---|---|
| 000000 | 00000 | $t2 | $t3 | 00100 | 100000 |

*Tabella 4: Esempio della disposizione di una SLL*

### Altre Eccezioni:

Dato che il campo funct è fornito solo di 6 bit, si è osservato che estendendo l'ISA questi erano troppo pochi. Per questo nel MIPS esteso, alcune operazioni di tipo "R" vanno ad utilizzare sia il campo opcode che il campo funct.

Questo tipo di istruzioni non vengono però implementate in questo progetto.

## Istruzioni di Tipo I:

| **Tipo di istruzione** | **Bit 31-26** | **Bit 25-21** | **Bit 20-16** | **Bit 15-11** | **Bit 10-6** | **Bit 5-0** |
|---|---|---|---|---|---|---|
| **Tipo-I** | opcode | rs | rt | imm | imm | imm |

*Tabella 5: Illustrazione della disposizione dei bit nelle istruzioni di tipo "I"*

Nelle istruzioni di tipo "I" sono presenti i campi "opcode", che qui identificano direttamente l'operazione da eseguire, il campo "rs" che rappresenta l'indirizzo del registro che contiene uno degli operandi, il campo "rt" che rappresenta il registro su cui salvare il dato elaborato e 21bit rappresentanti un numero (il tipo dato dipende dall'istruzione) che viene usato come altro operando. Il flusso generale di questo tipo di istruzioni è dunque il seguente:

$$RT = RS - operazione - Immediato$$

*ESEMPIO:*

$$ADDI\ \$t3, \$t2\ 10$$

| opcode | rs | rt | imm |
|---|---|---|---|
| 001000 | $t2 | $t3 | 10(decimale) |

*Tabella 6: Esempio della disposizione di una ADDI*

## Istruzioni di *Load* e *Store*:

Nel caso di istruzioni di *load* e di *store*, il flusso dell'istruzione rimane invariato, ma è interessante osservare che l'immediato viene usato per calcolare lo spiazzamento che deve avere l'indirizzo della memoria dati a cui si deve accedere, rispetto al contenuto del registro *RS.*

## Istruzioni di Salto Condizionato:

Seppure non propriamente un'eccezione, è interessante menzionare che nel MIPS le istruzioni di salto condizionato vengono gestite come una normale istruzione di tipo "I". Questa operazione genera un segnale che permette all'unità di controllo del flusso di eseguire un salto andando all'indirizzo di memoria indicato come immediato (l'immediato rappresenta lo spiazzamento in numero di parole rispetto all'istruzione successiva, per essere usato come indirizzo, l'immediato deve essere moltiplicato per quattro, ovvero, shiftato di due bit a sinistra).

## Istruzioni di Tipo J:

| Tipo di istruzione | Bit 31-26 | Bit 25-21 | Bit 20-16 | Bit 15-11 | Bit 10-6 | Bit 5-0 |
|---|---|---|---|---|---|---|
| **Tipo-J** | opcode | address | address | address | address | address |

*Tabella 7: Illustrazione della disposizione dei bit nelle istruzioni di tipo "J"*

Le istruzioni di tipo “J” sono utilizzate per la gestione dei salti incondizionati. Utilizzano solo il campo “opcode” e 26bit per identificare l’indirizzo di memoria assoluto (non c’è bisogno di fare nessuna operazione sull’indirizzo).

*ESEMPIO:*

$$J\ 10$$

| opcode | address |
|---|---|
| 000010 | 10(Dec) |

*Tabella 8: Esempio della disposizione di una J*

## Istruzioni Implementate:

| Mnemonic | Format | Description | Opcode (6b) | Funct (6b) | Notes |
|---|---|---|---|---|---|
| addu | R | rd = rs + rt (no overflow) | 000000 | 100001 | OK |
| add | R | rd = rs + rt (overflow, treated unsigned) | 000000 | 100000 | ⚠ implemented as unsigned |
| subu | R | rd = rs - rt (no overflow) | 000000 | 100011 | OK |
| sub | R | rd = rs - rt (overflow, treated unsigned) | 000000 | 100010 | ⚠ implemented as unsigned |
| and | R | rd = rs & rt | 000000 | 100100 | OK |
| or | R | rd = rs \| rt | 000000 | 100101 | OK |
| xor | R | rd = rs ^ rt | 000000 | 100110 | OK |
| nor | R | rd = ~(rs \| rt) | 000000 | 100111 | OK |
| sll | R | rd = rt << shamt | 000000 | 000000 | OK |
| srl | R | rd = rt >> shamt (logical) | 000000 | 000010 | OK |
| sltu | R | rd = (rs < rt) (unsigned) | 000000 | 101011 | OK |
| slt | R | rd = (rs < rt) (signed, as unsigned) | 000000 | 101010 | ⚠ implemented as unsigned |
| addi | I | rt = rs + imm (overflow, treated unsigned) | 001000 | - | ⚠ implemented as unsigned |
| addiu | I | rt = rs + imm (no overflow) | 001001 | 000000 | OK |
| andi | I | rt = rs & imm | 001100 | 000000 | OK |
| ori | I | rt = rs \| imm | 001101 | 000000 | OK |
| xori | I | rt = rs ^ imm | 001110 | 000000 | OK |
| lui | I | rt = imm << 16 | 001111 | 000000 | OK |
| sltiu | I | rt = (rs < imm) (unsigned) | 001011 | 000000 | OK |
| slti | I | rt = (rs < imm) (signed, as unsigned) | 001010 | 000000 | ⚠ implemented as unsigned |
| lw | I | rt = Mem[rs + offset] | 100011 | 000000 | OK |
| sw | I | Mem[rs + offset] = rt | 101011 | 000000 | OK |
| beq | I | if (rs == rt) branch | 000100 | 000000 | OK |
| bne | I | if (rs != rt) branch | 000101 | 000000 | OK |
| j | J | Jump to target | 000010 | 000000 | OK |

*Tabella 9: Elenco delle istruzioni implementate in questo progetto*

Nella tabella sopra riportata ci sono tutte le istruzioni implementate in questo progetto. Con OK si indica che l'istruzione è implementata in maniera nativa.

# Limitazioni di *TinyTapeout*:

*TinyTapeout* è un progetto che permette, utilizzando software e tools *open source* di fabbricare circuiti integrati condividendo l'area dell'integrato tra vari progettisti. Il singolo progettista deve quindi tenere in considerazione due fattori:

- Minimizzazione dell'area impiegata;
- L'inalterabilità del ***BUS-I/O.***

Nel caso di questo progetto il vincolo più limitante è proprio il secondo, in quanto il processore ha un'architettura 32bit, mentre *TinyTapeout* prevede solo:

- Otto pin d'ingresso;
- Otto pin d'uscita;
- Otto pin bidirezionali.

Questo implica che in questo progetto, oltre che avere il processore MIPS, deve anche avere un serializzatore che costruisce l'istruzione o la parola prelevata dalla memoria principale 8 bit alla volta. Di seguito viene lasciata una tabella riassuntiva della *pinout*.

| # | Input | Output | Bidirectional |
|---|---|---|---|
| **0** | instruction[0] | data_out[0] | address_out[0] |
| **1** | instruction[1] | data_out[1] | address_out[1] |
| **2** | instruction[2] | data_out[2] | address_out[2] |
| **3** | instruction[3] | data_out[3] | address_out[3] |
| **4** | instruction[4] | data_out[4] | address_out[4] |
| **5** | instruction[5] | data_out[5] | address_out[5] |
| **6** | instruction[6] | data_out[6] | address_out[6] |
| **7** | instruction[7] | data_out[7] | address_out[7] |

*Tabella 10: Pinout di TinyTapeout*

# Schema a Blocchi:

Si procede adesso a descrivere il sistema partendo da un excursus sul sistema globale, per poi entrare nel dettaglio di tutti quei blocchi più particolari che meritano di essere descritti in modo dettagliato.

## Dispositivo HOST:

Prima di procedere però, bisogna specificare che questo processore **NON** può operare da solo, perché, per motivi che si descriveranno in seguito, alcune funzionalità che normalmente sono responsabilità del processore stesso, non sono implementate all'interno di questo progetto. Per operare questo processore, sarà dunque necessario avere un dispositivo HOST (che può anche essere un microcontrollore) che avrà la responsabilità di gestire:

- Le memorie (programmi e dati);
- Il flusso del programma.

Di seguito viene illustrato il collegamento tra il processore ed il dispositivo HOST.

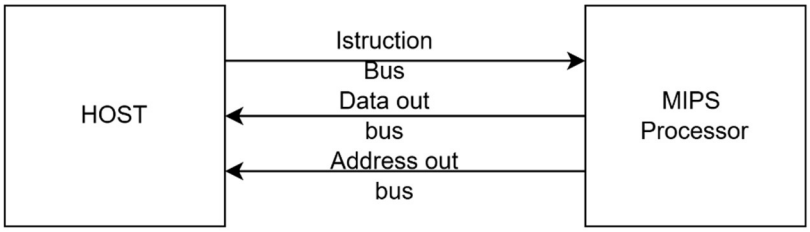



*Figura 1: Interconnessione tra HOST e Processore*

## Descrizione Generale:

Partendo dallo schema canonico del processore MIPS monociclo:

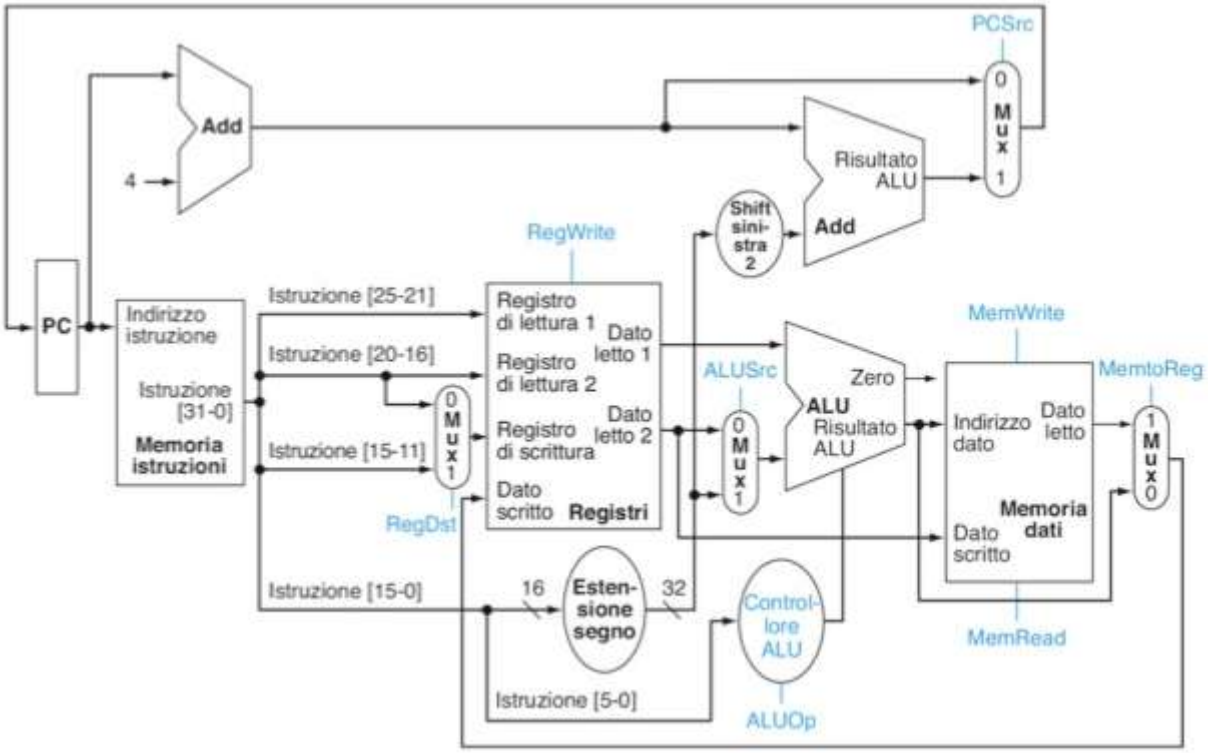



*Figura 2: CPU MIPS Monociclo*

Bisogna, alla luce delle limitazioni elencate nella sezione precedente, implementare questo schema con dei compromessi:

- La memoria istruzioni e programmi, per ragioni di utilizzabilità della struttura e di minimizzazione dell'area, devono essere implementate nel dispositivo *HOST*;

- Non essendoci la memoria programmi, anche il *program counter* e tutti organi che gestiscono il flusso delle istruzioni sono anch'esse implementate nel dispositivo *HOST.*

Aggiungendo anche il blocco serializzazione del dato, si illustra di seguito il sistema complessivo:

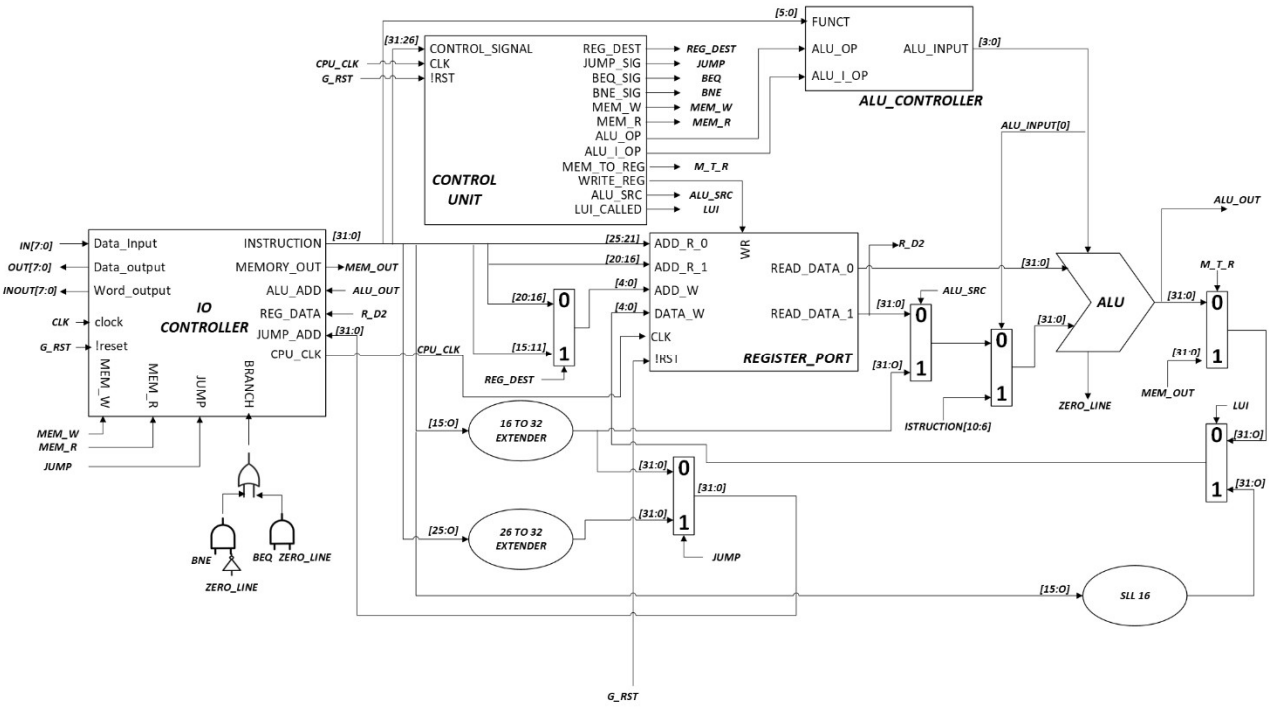



*Figura 3: Schema a Blocchi del Sistema.*

Procedendo a fare una panoramica dei blocchi implementati:

- **IO CONTROLLER:** è il blocco di serializzazione del dato. In seguito, vi è una sezione dedicata a questo blocco, per la descrizione generale del funzionamento di questa CPU basta sapere che ISTRUCTION e MEMORY_OUT sono correttamente riempite quando la CPU lo richiede;

- **CONTROL_UNIT:** è il blocco che gestisce tutto il *data path della CPU;*

- **ALU_CONTROLLER:** è il blocco che gestisce le operazioni che deve eseguire la **ALU**;

- **REGISTER_PORT:** è il blocco che al suo interno ha i registri del processore e gestisce lettura e salvataggio dei dati.

## Segnali della CONTROL_UNIT:

Di seguito viene riportata una tabella con i segnali d'uscita della *CONTROL_UNIT* ed una breve descrizione:

| USCITA | DESCRIZIONE |
|---|---|
| REG_DEST | Decide da quale campo dell'istruzione prelevare l'indirizzo del registro da scrivere: Se "0" si preleva il campo "RT", se "1" si preleva il campo "RD". |
| JUMP_SIG | Segnale che viene alzato se l'istruzione decodificata è un'istruzione di salto non condizionato. Necessario al funzionamento dell'*IO_CONTROLLER.* |
| BEQ_SIG | Segnale che viene alzato se l'istruzione decodificata è un'istruzione di salto Condizionato (BEQ). Necessario al funzionamento dell'*IO_CONTROLLER.* |
| BNE_SIG | Segnale che viene alzato se l'istruzione decodificata è un'istruzione di salto Condizionato (BNE). Necessario al funzionamento dell'*IO_CONTROLLER.* |
| MEM_W | Segnale che viene alzato se l'istruzione decodificate richiede una scrittura della Memoria dati situata nel dispositivo HOST. Necessario al funzionamento dell'*IO_CONTROLLER.* |
| MEM_R | Segnale che viene alzato se l'istruzione decodificate richiede una lettura della Memoria dati situata nel dispositivo HOST. Necessario al funzionamento dell'*IO_CONTROLLER.* |
| ALU_OP | BUS, manda l'informazione dell'operazione da eseguire all'*ALU_CONTROLLER* Nel caso di istruzioni di tipo "R". |
| ALU_I_OP | BUS, manda l'informazione dell'operazione da eseguire all'*ALU_CONTROLLER* Nel caso di istruzioni di tipo "I". |
| MEM_TO_REG | Decide da dove proviene il dato che deve essere scritto nel registro. Se "0" il dato proviene dalla ALU, se "1" il dato proviene dalla memoria dati contenuta nel Dispositivo HOST. |
| WRITE_REG | Abilita la scrittura del registro. |
| ALU_SRC | Decide da dove proviene il secondo operando della ALU. Se "0" questo proviene dal registro, se "1" proviene dall'immediato con estensione di segno. |
| LUI_CALLED | Segnale che viene alzato se l'istruzione decodificata un caricamento di un immediato (LUI). Necessario al funzionamento dell'*IO_CONTROLLER.* |

*Tabella 11: Descrizione dei segnali d'uscita della CONTROL_UNIT*

Si procede quindi a descrivere il flusso di funzionamento per ogni tipo d'istruzione implementato in questo *istruction set.*

## Funzionamento Istruzioni Standard di tipo “R”:

Osservando come il tipo di istruzione ne formatti il contenuto (Tabella 1) si può dedurre facilmente il funzionamento di un’istruzione standard di tipo “R”.

Una volta osservato che *l’opcode* è “0”, la *CONTROL_UNIT* imposta le sue uscite nel seguente modo:

| USCITA | STATO |
|---|---|
| REG_DEST | 1 |
| JUMP_SIG | 0 |
| BEQ_SIG | 0 |
| BNE_SIG | 0 |
| MEM_W | 0 |
| MEM_R | 0 |
| ALU_OP | *[1] |
| ALU_I_OP | * |
| MEM_TO_REG | 0 |
| WRITE_REG | 1 |
| ALU_SRC | 0 |
| LUI_CALLED | 0 |

*Tabella 12: Stato dei segnali d'uscita della CONTROL UNIT per istruzioni di tipo “R”*

Quindi:

- ***REGISTER_PORT*** *è* abilitato alla scrittura (*WRITE_REG 1*).
  Presenta in uscita il campo RS*(istruction*[25:21]) e RT *(istruction*[20:16]) rispettivamente alle porte *READ_DATA_0* e *READ_DATA_1*.
  Essendo *REG_DEST* abilitato, significa che l’indirizzo su cui scrivere il risultato viene preso dal campo RD dell’istruzione (*istruction*[15:11]), mentre il dato da scrivere viene presa dall’uscita dell’ALU.

- **ALU_CONTROLLER** legge e comanda all’ALU di eseguire il campo *function* dell’istruzione (*istruction*[5:0]).

## Istruzioni di *Shift*:

Nel caso di istruzioni di *Shift* il flusso di funzionamento è molto simile, l’unica cosa che cambia è che l’ALU prende come secondo operando il campo *shamt* dell’istruzione (*istruction* [10:4]). Questo cambio di input viene fatto grazie al bit *ALU_INPUT*[0] che nelle operazioni di *shift* vale “1” e che abilita il MUX che va in ingresso alla seconda porta dell’ALU.

[1] Per valutare al meglio ALU_OP e ALU_IOP analizzare il codice sorgente *control_unit_module.v*

## Funzionamento Istruzioni Standard di tipo “I”:

Osservando come il tipo di istruzione ne formatti il contenuto (Tabella 1) si può dedurre facilmente il funzionamento di un’istruzione standard di tipo “I”.

Una volta osservato che *l’opcode* è diverso da zero, questo viene decodificato dalla *CONTROL_UNIT* che nelle operazioni standard imposta le uscite nel seguente modo:

| USCITA | STATO |
|---|---|
| REG_DEST | 0 |
| JUMP_SIG | 0 |
| BEQ_SIG | 0 |
| BNE_SIG | 0 |
| MEM_W | 0 |
| MEM_R | 0 |
| ALU_OP | * |
| ALU_I_OP | * |
| MEM_TO_REG | 0 |
| WRITE_REG | 1 |
| ALU_SRC | 1 |
| LUI_CALLED | 0 |

*Tabella 13: Stato dei segnali d'uscita della CONTROL UNIT per istruzioni di tipo "I"*

Quindi:

- ***REGISTER_PORT*** *è* abilitato alla scrittura (*WRITE_REG 1*).
  Presenta in uscita il campo RS*(istruction*[25:21]) e l’immediato(istruction[15:0] con il segno esteso a 32 bit).
  Essendo *REG_DEST* disabilitato, significa che l’indirizzo su cui scrivere il risultato viene preso dal campo RT dell’istruzione (*istruction*[21:16]), mentre il dato da scrivere viene presa dall’uscita dell’ALU.

- **ALU_CONTROLLER** comanda all’ALU di eseguire il comando proveniente dalla *CONTROL_UNIT: ALU_I_OP*.

## Funzionamento delle Istruzioni di Load e Store:

Nel caso di una *load* o di una *store*, il flusso è simile, la *ALU* deve eseguire una addizione per il calcolo dell'indirizzo di memoria a cui accedere. Ciò che cambia è che la *CONTROL_UNIT* oltre i segnali elencati nel normale funzionamento delle istruzioni di tipo "I", alza anche i segnali *MEM_W* (per la *Store Word*) e *MEM_R* (per la *Load_Word)*. Grazie a questi due segnali e all'uscita dell'ALU, l'IO_CONTROLLER sarà in grado di gestire correttamente l'accesso in memoria. Di seguito la tabella dei segnali d'uscita della CONTROL_UNIT:

| USCITA | STATO |
|---|---|
| REG_DEST | 0 |
| JUMP_SIG | 0 |
| BEQ_SIG | 0 |
| BNE_SIG | 0 |
| MEM_W | 1(SW) |
| MEM_R | 1(LW) |
| ALU_OP | * |
| ALU_I_OP | * |
| MEM_TO_REG | 0 |
| WRITE_REG | 1(LW) |
| ALU_SRC | 0 |
| LUI_CALLED | 0 |

*Tabella 14: Stato dei segnali d'uscita della CONTROL UNIT per istruzioni di LOAD/STORE*

Per quanto riguarda l'istruzione *LUI*, Si alza il segnale *LUI_CALLED* e va a bypassare la ALU andando, quindi, a scrivere l'immediato (traslato di 16bit a sinistra) direttamente nel registro.

I segnali d'uscita diventano dunque così:

| USCITA | STATO |
|---|---|
| REG_DEST | 0 |
| JUMP_SIG | 0 |
| BEQ_SIG | 0 |
| BNE_SIG | 0 |
| MEM_W | 0 |
| MEM_R | 0 |
| ALU_OP | * |
| ALU_I_OP | * |
| MEM_TO_REG | 0 |
| WRITE_REG | 1 |
| ALU_SRC | 0 |
| LUI_CALLED | 1 |

*Tabella 15: Stato dei segnali d'uscita della CONTROL UNIT per istruzioni di LOAD IMMIDIATE*

## Funzionamento delle Istruzioni di Salto Condizionato:

Nelle istruzioni di salto condizionato, i processori MIPS vanno a valutare se la differenza tra i contenuti dei registri comparati vale zero nel caso di una BEQ o sia diverso da zero nel caso di una BNE.

L'indirizzo a cui saltare viene scritto nella porta del *CONTROLLER_IO JUMP_ADD*.

La *zero_line* viene data in ingresso ad una semplice rete combinatoria che, indipendentemente dal tipo di salto condizionato, da come risultato "1" se la condizione del salto è soddisfatta, altrimenti "0".

I segnali d'uscita della *CONTROL_UNIT sono dunque questi:*

| USCITA | STATO |
|---|---|
| REG_DEST | 1 |
| JUMP_SIG | 0 |
| BEQ_SIG | 1(BEQ) |
| BNE_SIG | 1(BNE) |
| MEM_W | 0 |
| MEM_R | 0 |
| ALU_OP | * |
| ALU_I_OP | * |
| MEM_TO_REG | 0 |
| WRITE_REG | 0 |
| ALU_SRC | 0 |
| LUI_CALLED | 0 |

*Tabella 16 Stato dei segnali d'uscita della CONTROL UNIT per istruzioni di salto condizionato*

## Funzionamento delle Istruzioni di Tipo “J”:

Osservando come il tipo di istruzione ne formatti il contenuto (Tabella 1) si può dedurre facilmente il funzionamento di un’istruzione standard di tipo “J”.

Una volta osservato che *l’opcode* è diverso da zero, questo viene decodificato dalla *CONTROL_UNIT* che nelle operazioni standard imposta le uscite nel seguente modo:

| USCITA | STATO |
|---|---|
| REG_DEST | 0 |
| JUMP_SIG | 1 |
| BEQ_SIG | 0 |
| BNE_SIG | 0 |
| MEM_W | 0 |
| MEM_R | 0 |
| ALU_OP | 0 |
| ALU_I_OP | 0 |
| MEM_TO_REG | 0 |
| WRITE_REG | 0 |
| ALU_SRC | 0 |
| LUI_CALLED | 0 |

*Tabella 17: Stato dei segnali d'uscita della CONTROL UNIT per istruzioni di salto incondizionato*

Quindi:

- JUMP_SIGNAL asserito, permette al *CONTROLLER_IO* di gestire correttamente il salto incondizionato. Lo stesso segnale permette di far scrivere alla porta *JUMP_ADD* i 26bit estesi fino a 32bit scritti nell’istruzione (*istruction[26:0]).*

## Descrizione Dettagliata dei Blocchi:

In questa sezione si procede a spiegare nel dettaglio il funzionamento dei blocchi *IO_CONTROLLER* e del *REGISTER_PORT* (gli altri blocchi non sono altro che implementazioni di *switch case*). Per ragioni di semplicità di trattazione, non verrà riportato il codice HDL, ma verrà indicato il nome del file che contiene il modulo descritto in quanto quest'ultimo è liberamente consultabile su GitHub.

### *IO_CONTROLLER:*[2]

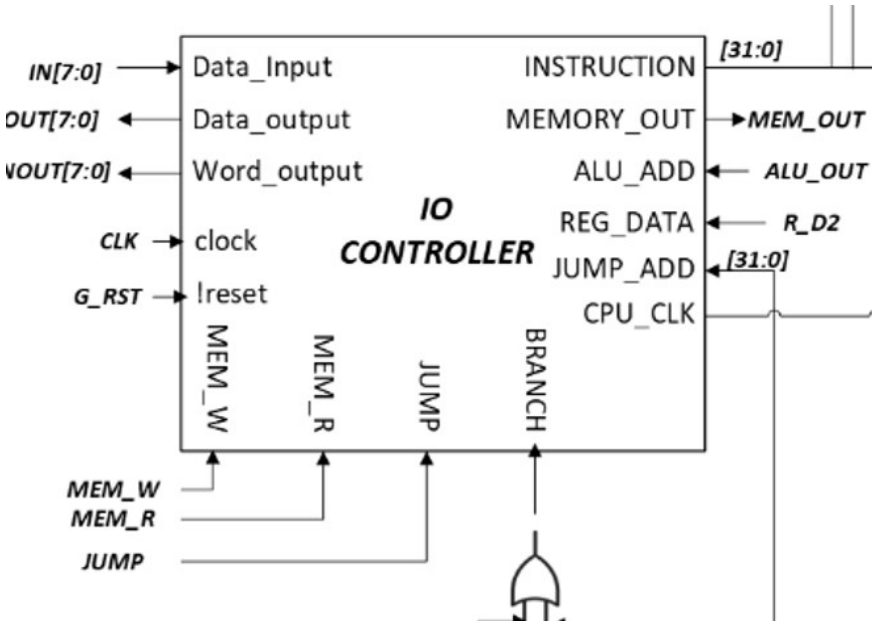



*Figura 4: IO_CONTROLLER*

*L'IO_CONTROLLER* è il blocco che permette l'interfacciamento della CPU (architettura a 32bit) verso l'IO di *TinyTapeout* (8bit).

È chiaro che c'è una fase di serializzazione in cui l'istruzione viene costruita otto bit la volta trasferendo i dati dal dispositivo *HOST* al *IO_CONTROLLER* passando per *DATA_IN*.

Quindi, in quattro colpi di clock (*rising edge*) l'istruzione viene completata e CPU_clk viene messo ad alto ed il processore può evolvere liberamente.

Da qui, a seconda dell'istruzione, *l'IO_CONTROLLER* esegue delle azioni che devono essere adeguatamente accolte dal dispositivo HOST.

[2] IO_controller_moduel.v

## Gestione delle Istruzioni Aritmetiche:

Nel caso delle istruzioni aritmetiche, siano esse di tipo R o di tipo I *l'IO_CONTROLLER* dopo un tempo di clock di stallo in cui abbassa l'uscita CPU_CLOCK per permettere la scrittura del registro modificato, è già pronto a ricevere una nuova istruzione

## Gestione delle Istruzioni di salto e di *Store Word*:

Sia nel caso di istruzioni di salto (condizionato e non) e di quelle di *store* il flusso di funzionamento è lo stesso, infatti è necessario, nel caso di salto (se la condizione dello stesso è rispettata, altrimenti viene trattata come un'istruzione aritmetica) bisogna portare al dispositivo *HOST* l'indirizzo a cui saltare, nel caso di salvataggio in memoria, bisogna portare sia l'indirizzo della memoria dove salvare la parola e la parola stessa.

Quindi, nel caso di salto, dopo un tempo di clock di stallo per permettere al dispositivo *HOST* di prepararsi ad accogliere i dati, l'*IO_CONTROLLER* fa uscire otto bit la volta:

- da *DATA_OUTPUT* l'indirizzo di memoria su cui salvare la parola nel caso di *store* oppure l'indirizzo della memoria programmi a cui saltare nel caso di salto;
- da *WORD_OUT* la parola che deve essere salvata nel caso di *store*.[3]

Dopodiché, **SOLO** nel caso di *store* vi è un tempo di clock di stallo per permettere al dispositivo *HOST* di salvare correttamente la parola in memoria.

## Gestione della *Load Word:*

Nel caso della *load word* si eseguono gli stessi passi dell'istruzione di *store word*, dove da *DATA_OUT* questa volta viene portato fuori l'indirizzo di memoria da cui leggere, ma poi c'è:

- Un tempo di clock di stallo per poter permetter al dispositivo *HOST* di accedere alla memoria
- WORD_OUT viene messo a "0xF" in modo da rendere più facile la gestione della scrittura della parola, quindi, ci sono quattro tempi di clock in cui viene trasferita la parola dal dispositivo *HOST* al *CONTROLLER_IO* attraverso *DATA_IN*

[3] Una possibile miglioria che non è ancora stata implementata, ma che prima della spedizione in fonderia del progetto verrà fatta è quella di poter leggere l'istruzione successiva mentre viene trasferita la parola.

- Un tempo di clock di stallo in cui viene disattivato *CPU_CLOCK* per permettere sia al dispositivo *HOST* di prelevare una nuova istruzione, sia alla CPU di salvare nel registro.

## REGISTER_PORT:

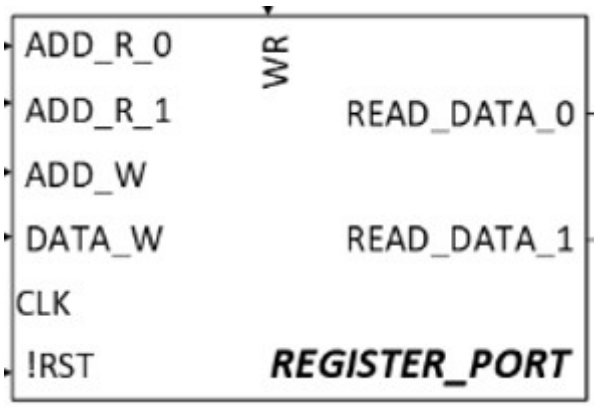



*Figura 5: REGISTER_PORT*

Il *REGISTER_PORT* è un blocco che gestisce la lettura e la scrittura dei registri del processore. La scrittura del registro avviene sul fronte negativo del segnale di clock e questa viene abilitata tramite un segnale di abilitazione *WR.*

Il registro ad indirizzo "0x00" non può essere modificato in quanto nelle architetture MIPS è riservato a contenere il valore "0" (è infatti indicato nel codice e *assembly* come $ZERO).

Quindi, il sistema funziona nel seguente modo:

- **ADD_R_0**: Si pone l'indirizzo del registro il cui contenuto si presenta sulla porta *READ_DATA_0;*
- **ADD_R_1**: Si pone l'indirizzo del registro il cui contenuto si presenta sulla porta *READ_DATA_1;*
- **ADD_W**: Si pone l'indirizzo del registro che deve essere scritto;
- **DATA_W**: Si pone la parola da scrivere;
- **CLK**: (è sul fronte negativo) abilita la scrittura;
- **!RST**: svuota il contenuto della memoria;
- **WR**: segnale di abilitazione della scrittura del registro.

## SIMULAZIONE E TEST_BENCH:

Oltre le varie simulazioni dei singoli sottosistemi, due sono le simulazioni RTL fondamentali. La prima è quella della sola CPU. In questa simulazione si sono andate ad eseguire tutte le istruzioni implementate nel progetto. Di seguito la stampa di una istruzione di *ADDI*. Da notare che gli ingressi della ALU sono visualizzati come esadecimale, mentre l'uscita come decimale.

```
# KERNEL: --- Segnali dopo ADDI $t0, $t2, 15 ---
# KERNEL: Instruction          : 00100001010010000000000000001111
# KERNEL: ALU Source A         : 0000002a
# KERNEL: ALU Source B (mux)   : 0000000f
# KERNEL: Extender output      : 0000000f
# KERNEL: ALU Control          : 0010
# KERNEL: ALU Output           :         57
# KERNEL: MUX Address Write    :  8
# KERNEL: Data to Write (MUX)  : 00000039
# KERNEL: LUI_MUX_output      : 00000000000000000000000000111001
# KERNEL: Control Signals      : 011011000000
# KERNEL:  [11] LUI_CALLED     : 0
# KERNEL:   [10] ALU_SRC        : 1
# KERNEL:   [9]  WRITE_REG      : 1
# KERNEL:   [8]  MEM_TO_REG     : 0
# KERNEL:   [7]  ALU_OP_1       : 1
# KERNEL:   [6]  ALU_OP_0       : 1
# KERNEL:   [5]  MEM_READ       : 0
# KERNEL:   [4]  MEM_WRITE      : 0
# KERNEL:   [3]  BNE_SIGNAL     : 0
# KERNEL:   [2]  BEQ_SIGNAL     : 0
# KERNEL:   [1]  JUMP_SIGNAL    : 0
# KERNEL:   [0]  REG_DEST       : 0
# KERNEL: Mem Write            : 0
# KERNEL: Mem Read             : 0
# KERNEL: Address Out          : 00000039
# KERNEL: Register Data (rt)   : 00000039
# KERNEL: Jump Address         : 0148000f
# KERNEL: Branch Success       : 0
# KERNEL: Jump                 : 0
# KERNEL: ------------------------
```

Questa simulazione è stata fondamentale per osservare che l'*istruction set* fosse correttamente implementato.

La seconda simulazione è quella del sistema globale. Questa si focalizza maggiormente sull'interazione tra l'*IO_CONTROLLER* ed il resto della CPU. In particolare, in questa simulazione viene eseguita una *load word* , in cui si vuole caricare sui registri la parla scritta in esadecimale "***DEADBEEF***".

```
# KERNEL: monitor : T=5000 | instr=00000000 | addr_out=00 |
mem_out=00000000 | mem_r=x | mem_w=x | ALU_out=00000000
# KERNEL: PREPARE_INSTRUCTION
# KERNEL: PREPARE_INSTRUCTION
# KERNEL: |clk=1| FETCH_INST | count=0 | byte_in=00000100 |
instruction(next)=00000000000000000000000000000100
# KERNEL: |clk=0| FETCH_INST | count=0 | byte_in=00000100 |
instruction(next)=00000000000000000000000000000100
# KERNEL: |clk=0| FETCH_INST | count=0 | byte_in=00000000 |
instruction(next)=00000000000000000000000000000000
# KERNEL: |clk=1| FETCH_INST | count=0 | byte_in=00000000 |
instruction(next)=00000000000000000000000000000000
# KERNEL: |clk=1| FETCH_INST | count=1 | byte_in=00000000 |
instruction(next)=00000000000000000000000000000000
# KERNEL: |clk=0| FETCH_INST | count=1 | byte_in=00000000 |
instruction(next)=00000000000000000000000000000000
# KERNEL: |clk=0| FETCH_INST | count=1 | byte_in=00001000 |
instruction(next)=00000000000000000000100000000000
# KERNEL: |clk=1| FETCH_INST | count=1 | byte_in=00001000 |
instruction(next)=00000000000000000000100000000000
# KERNEL: |clk=1| FETCH_INST | count=2 | byte_in=00001000 |
instruction(next)=00000000000010000000100000000000
# KERNEL: monitor : T=35000 | instr=00000800 | addr_out=00 |
mem_out=00000000 | mem_r=x | mem_w=x | ALU_out=00000000
# KERNEL: |clk=0| FETCH_INST | count=2 | byte_in=00001000 |
instruction(next)=00000000000010000000100000000000
# KERNEL: |clk=0| FETCH_INST | count=2 | byte_in=10001101 |
instruction(next)=00000001000110100001000000000000
# KERNEL: |clk=1| FETCH_INST | count=2 | byte_in=10001101 |
instruction(next)=00000001000110100001000000000000
# KERNEL: |clk=1| FETCH_INST | count=3 | byte_in=10001101 |
instruction(next)=10001101100011010001000000000000
# KERNEL: monitor : T=45000 | instr=008d0800 | addr_out=00 |
mem_out=00000000 | mem_r=x | mem_w=x | ALU_out=00000000
# KERNEL: |clk=0| FETCH_INST | count=3 | byte_in=10001101 |
instruction(next)=10001101100011010001000000000000
# KERNEL: |clk=1| FETCH_INST | count=3 | byte_in=10001101 |
instruction(next)=10001101100011010001000000000000
# KERNEL: |clk=1| FETCH_INST | count=4 | byte_in=10001101 |
instruction(next)=10001101100011010001000000000000
# KERNEL: monitor : T=55000 | instr=8d8d0800 | addr_out=00 |
mem_out=00000000 | mem_r=x | mem_w=x | ALU_out=00000000
# KERNEL: |clk=0| FETCH_INST | count=4 | byte_in=10001101 |
instruction(next)=10001101100011010001000000000000
# KERNEL: |clk=1| FETCH_INST | count=4 | byte_in=10001101 |
instruction(next)=10001101100011010001000000000000
# KERNEL: EXECUTE, count=5
# KERNEL: --LW has been called --
```

```
# KERNEL: --ALU_OP_CALLED: 0
# KERNEL: EXECUTE, count=5
# KERNEL: monitor : T=65000 | instr=8d8d0800 | addr_out=00 |
mem_out=00000000 | mem_r=1 | mem_w=0 | ALU_out=00000800
# KERNEL: EXECUTE, count=5
# KERNEL: EXECUTE, count=5
```

Viene poi prelevato l'indirizzo di memoria da cui prelevare la parola:

```
# KERNEL: |clk=1| GET_WORD | count=0 | word=00000000 | addr_in=00000800 |
data_out(next)=00 | address_out(next)=00
# KERNEL: |clk=0| GET_WORD | count=0 | word=00000000 | addr_in=00000800 |
data_out(next)=00 | address_out(next)=00
# KERNEL: |clk=1| GET_WORD | count=0 | word=00000000 | addr_in=00000800 |
data_out(next)=00 | address_out(next)=00
# KERNEL: |clk=1| GET_WORD | count=1 | word=00000000 | addr_in=00000800 |
data_out(next)=00 | address_out(next)=08
# KERNEL: |clk=0| GET_WORD | count=1 | word=00000000 | addr_in=00000800 |
data_out(next)=00 | address_out(next)=08
# KERNEL: |clk=1| GET_WORD | count=1 | word=00000000 | addr_in=00000800 |
data_out(next)=00 | address_out(next)=08
# KERNEL: |clk=1| GET_WORD | count=2 | word=00000000 | addr_in=00000800 |
data_out(next)=00 | address_out(next)=00
# KERNEL: monitor : T=95000 | instr=8d8d0800 | addr_out=08 |
mem_out=00000000 | mem_r=1 | mem_w=0 | ALU_out=00000800
# KERNEL: |clk=0| GET_WORD | count=2 | word=00000000 | addr_in=00000800 |
data_out(next)=00 | address_out(next)=00
# KERNEL: |clk=1| GET_WORD | count=2 | word=00000000 | addr_in=00000800 |
data_out(next)=00 | address_out(next)=00
# KERNEL: |clk=1| GET_WORD | count=3 | word=00000000 | addr_in=00000800 |
data_out(next)=00 | address_out(next)=00
# KERNEL: monitor : T=105000 | instr=8d8d0800 | addr_out=00 |
mem_out=00000000 | mem_r=1 | mem_w=0 | ALU_out=00000800
# KERNEL: |clk=0| GET_WORD | count=3 | word=00000000 | addr_in=00000800 |
data_out(next)=00 | address_out(next)=00
# KERNEL: |clk=1| GET_WORD | count=3 | word=00000000 | addr_in=00000800 |
data_out(next)=00 | address_out(next)=00
# KERNEL: STALL
# KERNEL: STALL
# KERNEL: STALL
```

Viene poi scritta la parola "***DEADBEEF***":

```
# KERNEL: |clk=1| WRITE_WORD | count=0 | byte_in=8d |
memory_out(next)=0000008d
# KERNEL: [125000] STALL visto: address_out=0xFF. In attesa dell'avvio
scrittura (addr=0x01)...
# KERNEL: monitor : T=125000 | instr=8d8d0800 | addr_out=ff |
```

```
mem_out=00000000 | mem_r=1 | mem_w=0 | ALU_out=00000800
# KERNEL: |clk=0| WRITE_WORD | count=0 | byte_in=8d |
memory_out(next)=0000008d
# KERNEL: |clk=0| WRITE_WORD | count=0 | byte_in=ef |
memory_out(next)=000000ef
# KERNEL: |clk=1| WRITE_WORD | count=0 | byte_in=ef |
memory_out(next)=000000ef
# KERNEL: |clk=1| WRITE_WORD | count=1 | byte_in=ef |
memory_out(next)=0000efef
# KERNEL: monitor : T=135000 | instr=8d8d0800 | addr_out=ff |
mem_out=000000ef | mem_r=1 | mem_w=0 | ALU_out=00000800
# KERNEL: |clk=0| WRITE_WORD | count=1 | byte_in=ef |
memory_out(next)=0000efef
# KERNEL: |clk=0| WRITE_WORD | count=1 | byte_in=be |
memory_out(next)=0000beef
# KERNEL: |clk=1| WRITE_WORD | count=1 | byte_in=be |
memory_out(next)=0000beef
# KERNEL: |clk=1| WRITE_WORD | count=2 | byte_in=be |
memory_out(next)=00bebeef
# KERNEL: monitor : T=145000 | instr=8d8d0800 | addr_out=ff |
mem_out=0000beef | mem_r=1 | mem_w=0 | ALU_out=00000800
# KERNEL: |clk=0| WRITE_WORD | count=2 | byte_in=be |
memory_out(next)=00bebeef
# KERNEL: |clk=0| WRITE_WORD | count=2 | byte_in=ad |
memory_out(next)=00adbeef
# KERNEL: |clk=1| WRITE_WORD | count=2 | byte_in=ad |
memory_out(next)=00adbeef
# KERNEL: |clk=1| WRITE_WORD | count=3 | byte_in=ad |
memory_out(next)=adadbeef
# KERNEL: monitor : T=155000 | instr=8d8d0800 | addr_out=ff |
mem_out=00adbeef | mem_r=1 | mem_w=0 | ALU_out=00000800
# KERNEL: |clk=0| WRITE_WORD | count=3 | byte_in=ad |
memory_out(next)=adadbeef
# KERNEL: |clk=0| WRITE_WORD | count=3 | byte_in=de |
memory_out(next)=deadbeef
# KERNEL: |clk=1| WRITE_WORD | count=3 | byte_in=de |
memory_out(next)=deadbeef
# KERNEL: |clk=1| WRITE_WORD | count=4 | byte_in=de |
memory_out(next)=deadbeef
# KERNEL: monitor : T=165000 | instr=8d8d0800 | addr_out=ff |
mem_out=deadbeef | mem_r=1 | mem_w=0 | ALU_out=00000800
```

## Conclusioni:

Allo stato attuale, l'analisi RTL ha dimostrato il corretto funzionamento del sistema, questo è inoltre sintetizzabile, deve però essere ancora sviluppato l'analisi della STA e la LEC finale. Verrà però finalizzato il tutto prima del prossimo lancio di TinyTapeout.

## Link Utili:

MIPS Encoding Reference

HP_AppA.pdf (MIPS R2000 Assembly Language Encoding)